

오픈소스SW의 이해

CHAPTER 5

SEMESTER | 2026-2

박성미

The background image shows a wide-angle view of the Hanyang University campus. In the foreground, there is a large, modern building with a glass facade, likely the Hanyang University Library. Behind it, a large, open plaza with a fountain is visible. The plaza is surrounded by trees and other campus buildings. In the distance, a large hill (Hanyang Mountain) rises against a clear sky. The entire image is overlaid with a semi-transparent blue filter.

Oh My Git!

❖ Git 시각화 게임

<https://ohmygit.org/>

다운로드:

<https://blinry.itch.io/oh-my-git>

▼ 오늘

oh-my-git ✓

oh-my-git.pck

▼ 지난 달

dependencies



BackReloadToggle music

Living dangerously

So you have decided to apply for time travel school, to learn how to use this time machine called "Git"!

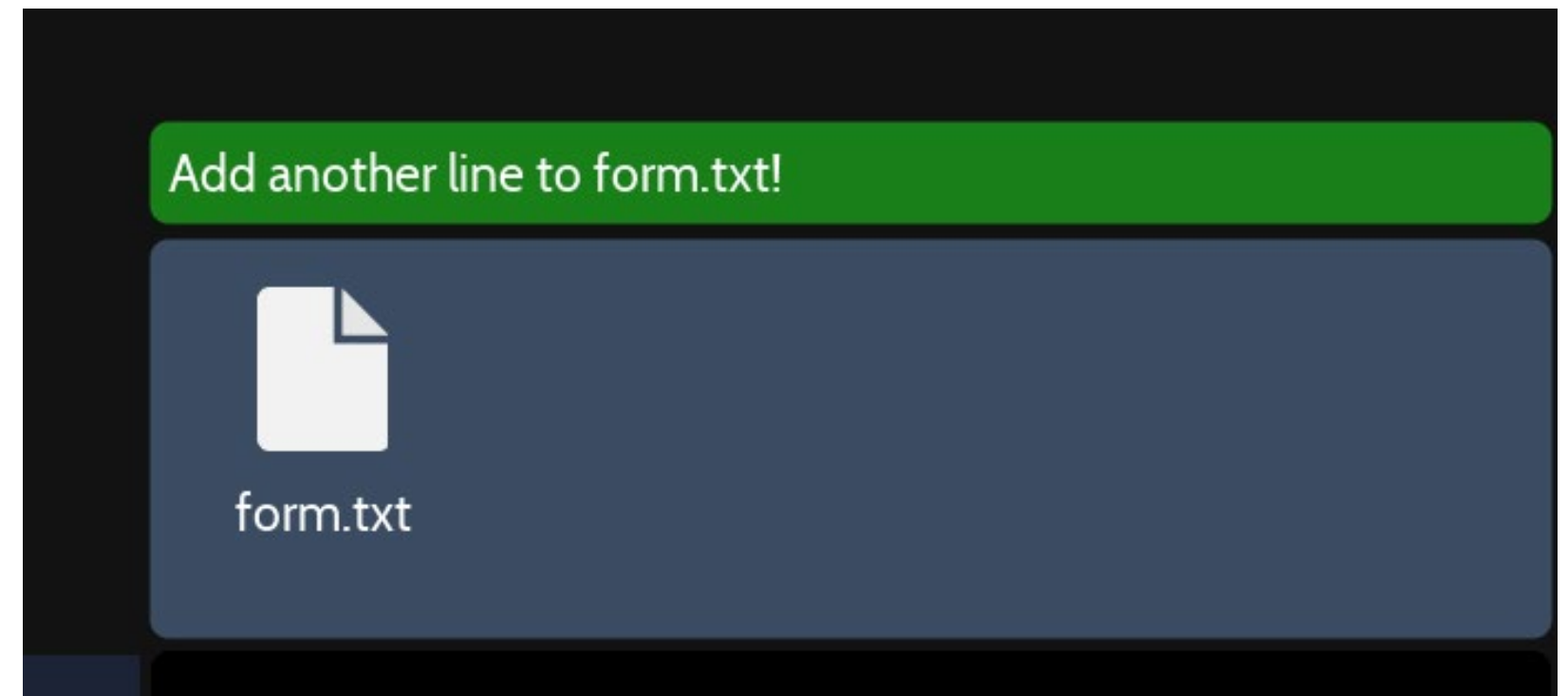
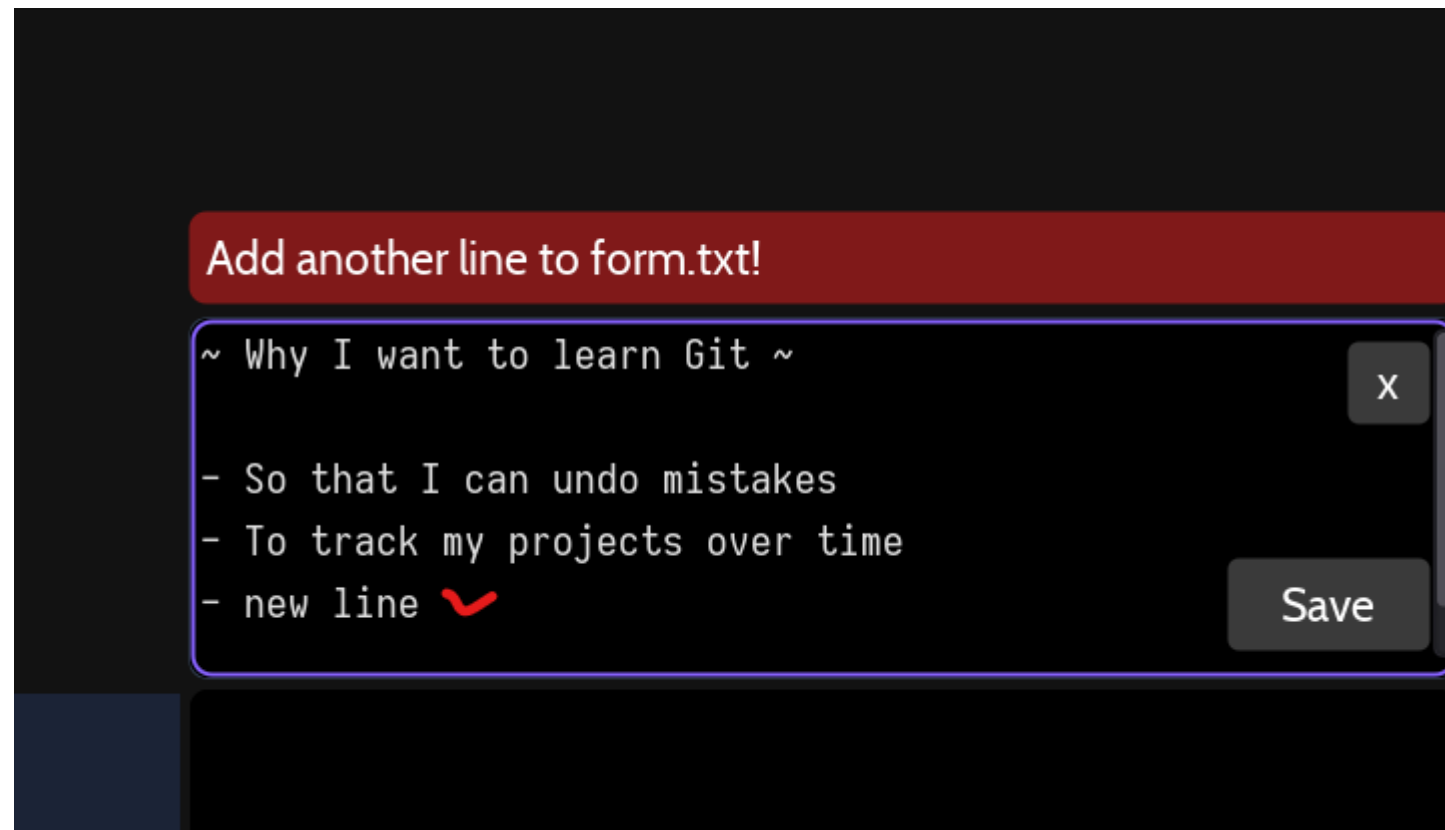
How exciting!

You're almost done with the paperwork! You just need to fill in one more reason why you want to learn Git.

Add another line to form.txt!

form.txt

Oh My Git



Back

Reload

Toggle music

intro

Living dangerously

Making backups

Enter the time machine

The command line

Your first commit

Working together

files

Unexpected Roommates

Interior design

branches

Moving through time

Make parallel commits

Creating branches

Branches grow with you!

Deleting branches

Moving branches around

merge

Merging timelines

Contradictions

index

Step by step

Add new files to the index

CLASSUM

논의하면서 풀기

- 1) 조별로 github id 공유
- 2) 팀장 선택 > 팀장은 public 레포지토리 생성
- 3) 조원들 collaborator로 추가
- 4) 팀장은 main 브랜치에 README.md, LICENSE, .gitignore 생성 및 커밋
- 5) 각 팀원은 레포지토리 clone
- 6) *팀원 별 브랜치 생성 (feat_githubid)*

```
git branch -m <변경전이름> <변경 후 이름>
git branch -m master main
```

7) 브랜치 1개 당 2명씩 작업 (충돌 발생용)

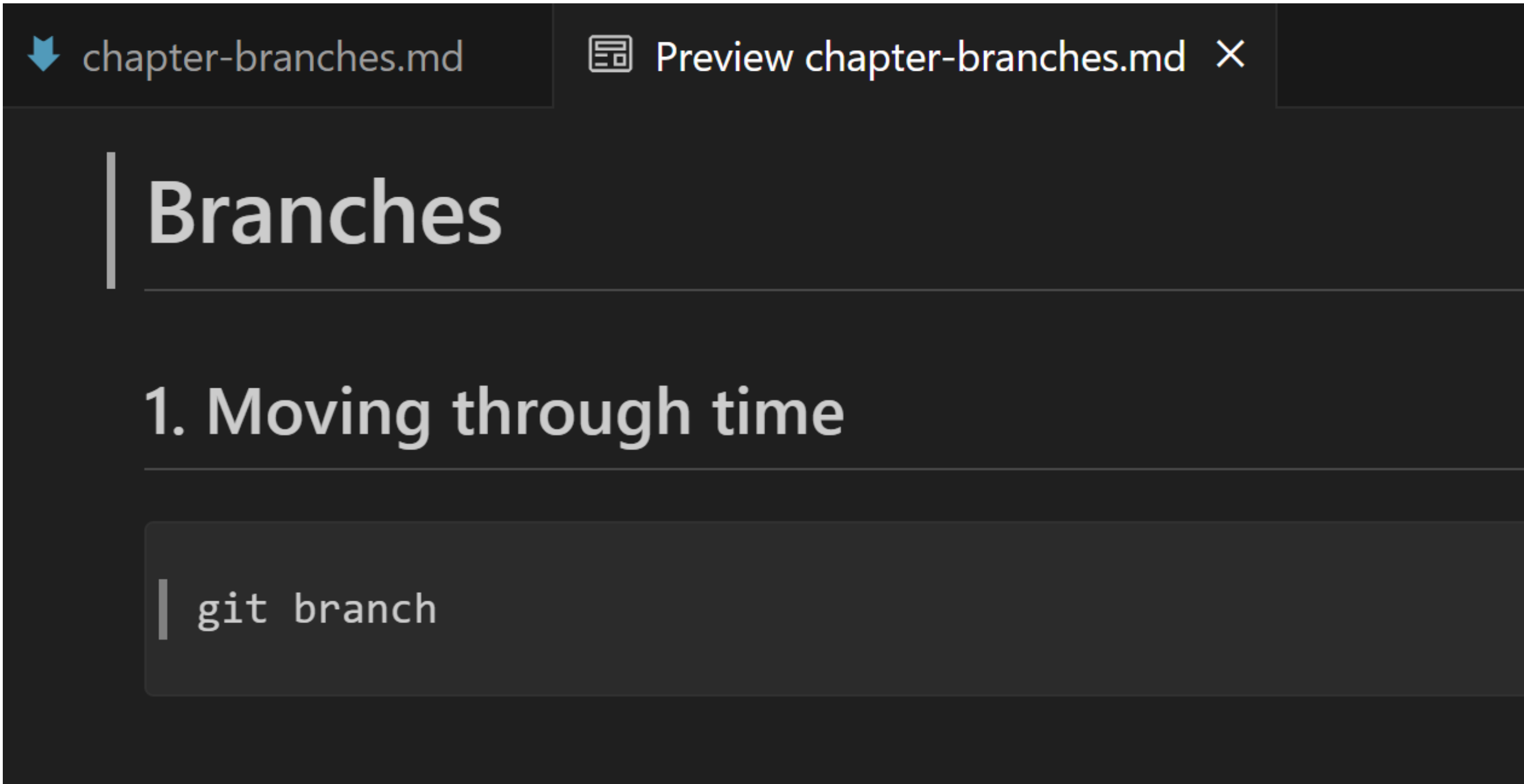
- 8) README.md에 학번 및 github ID 정리
- 9) README.md에 githubID 당 분담 문제 정리
- 10) Chapter별 md 생성 (chapter-branches.md, chapter-intro.md)
- 11) 문제 다 풀면 main으로 merge 요청
- 12) 팀장은 코드 검토 후 merge 수행

- GPT 사용 금지
- Git 명령어 검색 가능

CLASSUM

예)

chapter/branches



chapter-branches.md Preview chapter-branches.md X

Branches

1. Moving through time

```
git branch
```

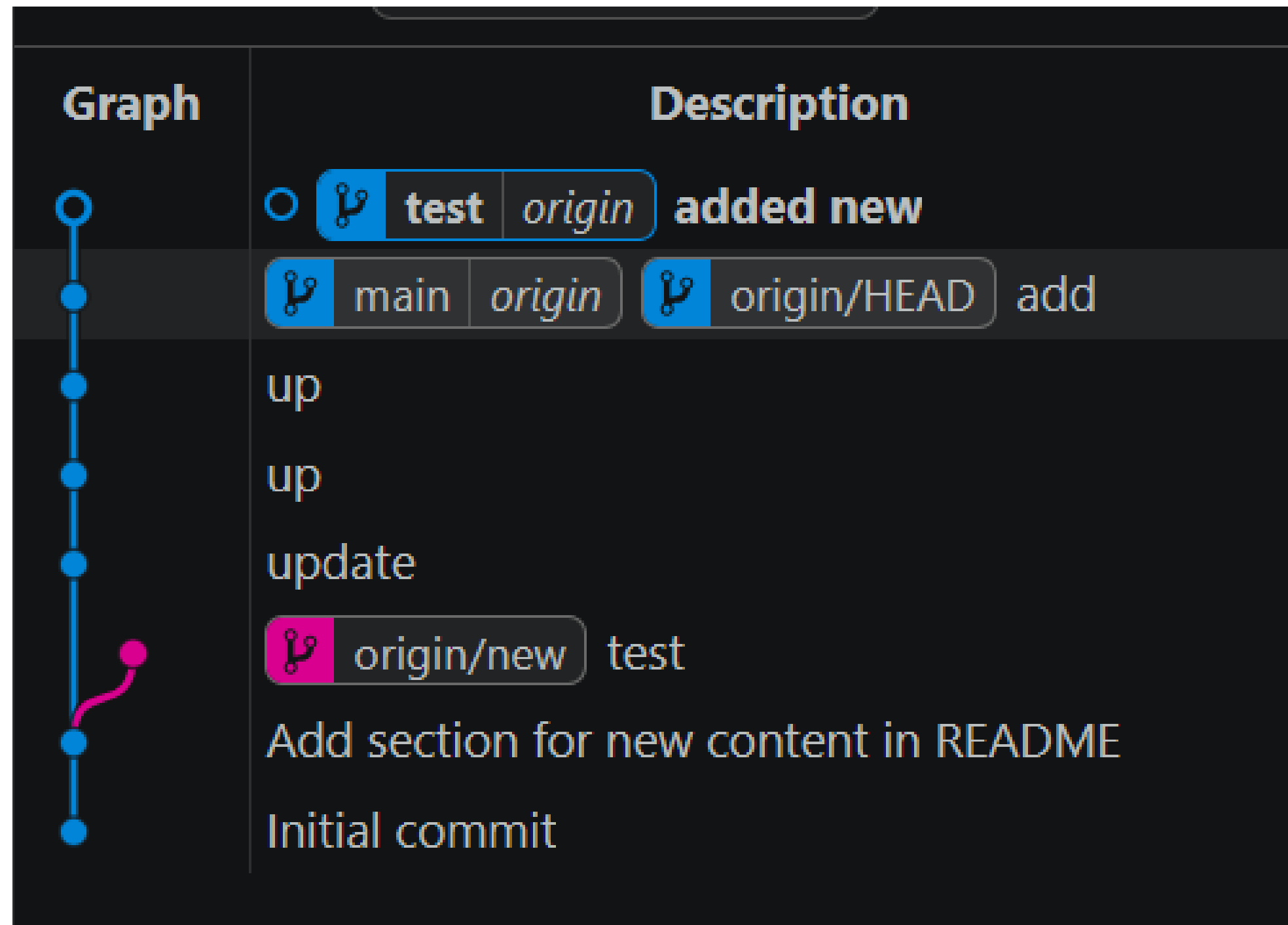
- PS F:\한림대\오픈소스SW의 이해\2025_opensource_test> git branch test
- PS F:\한림대\오픈소스SW의 이해\2025_opensource_test> git switch test
Switched to branch 'test'
- PS F:\한림대\오픈소스SW의 이해\2025_opensource_test> git add .
- PS F:\한림대\오픈소스SW의 이해\2025_opensource_test> git commit -m 'added new'
[test 1b3940e] added new
1 file changed, 1 insertion(+)
create mode 100644 new.py
- Ⓢ PS F:\한림대\오픈소스SW의 이해\2025_opensource_test> git push
fatal: The current branch test has no upstream branch.
To push the current branch and set the remote as upstream, use

git push --set-upstream origin test

To have this happen automatically for branches without a tracking upstream, see 'push.autoSetupRemote' in 'git help config'.

💡 PS F:\한림대\오픈소스SW의 이해\2025_opensource_test> git push -u origin test

LOCAL



test had recent pushes 3 seconds ago

Compare & pull request

main 2 Branches 0 Tags

Go to file T

Add file

Code

hallym add a11beb4 · last year 6 Commits

.gitignore	Initial commit	last year
LICENSE	Initial commit	last year
README.md	add	last year

README MIT license

팀원

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

base: main ← compare: test ✓ **Able to merge.** These branches can be automatically merged.

Add a title *

added new

Add a description

WritePreview

HBBI≡<>🔗⋮⋮⋮🔗@🗨️↶🗑️

new ~~py~~ added

Markdown is supportedPaste, drop, or click to add files

Create pull request

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

1 commit1 file changed1 contributor

Commits on Oct 8, 2026

added new

suz

committed 2 minutes ago

1b3940e

<>

Copilot can review pull requests

Request a review from Copilot to get fast, actionable feedback on your code, so you can start iterating before you receive a human review.

Learn more

OK, dismiss

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use [Closing keywords](#) in the description to automatically close issues

Helpful resources

[GitHub Community Guidelines](#)

팀장

added new #1

Open

outspark wants to merge 1 commit into main from test

Conversation 0

Commits 1

Checks 0

Files changed 1

outspark commented 11 minutes ago

Owner

...

new py added

Mention @copilot in a comment to make changes to this pull request.

added new

1b3940e

No conflicts with base branch

Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

added new #1

 Merged outspark merged 1 commit into `main` from `test`  now

 Conversation 0

 Commits 1

 Checks 0

 Files changed 1



outspark commented 12 minutes ago

Owner ...

new py added



 Mention @copilot in a comment to make changes to this pull request.



added new

1b3940e



outspark merged commit `21ea7f3` into `main` now

Revert



Pull request successfully merged and closed

You're all set — the `test` branch can be safely deleted.

Delete branch

Step 1 Clone the repository or update your local repository with the latest changes.

```
git pull origin main
```



Step 2 Switch to the base branch of the pull request.

```
git checkout main
```



Step 3 Merge the head branch into the base branch.

```
git merge test
```



Step 4 Push the changes.

```
git push -u origin main
```



작업 디렉터리

실제로 수정하는 파일

git add

스테이징 영역 (index)

다음 커밋에 넣을 변경

git commit

로컬 저장소

내 컴퓨터의 커밋 이력

git push

원격 저장소

GitHub 등의 공유 저장소

01 설정과 시작

커밋에 기록할 이름·이메일 설정

```
git config --global user.name "Name"
git config --global user.email "me@host.com"
```

--global: 내 계정의 모든 저장소에 적용. 생략하면 현재 저장소에만 적용.

현재 폴더를 Git 저장소로 초기화

```
git init
```

원격 저장소를 로컬로 복제

```
git clone <url>
```

02 변경을 기록하기

파일의 변경 내용을 스테이징

```
git add <file>
```

현재 폴더 아래의 변경을 모두 스테이징

```
git add .
```

새 파일·수정·삭제 모두 포함.

스테이징된 내용을 커밋으로 저장

```
git commit -m "message"
```

추적 중인 파일의 수정·삭제를 커밋

```
git commit -am "message"
```

-a: 자동 스테이징. 새 파일은 포함하지 않음.

이동·이름 변경 / 파일 삭제 후 스테이징

```
git mv <old> <new>
git rm <file>
```

파일은 유지하고 Git 추적만 제거

```
git rm --cached <file>
```

기본 용어

main: 관례적으로 쓰는 기본 브랜치 이름
origin: clone 시 기본 원격 이름
HEAD: 현재 체크아웃한 커밋을 가리킴
HEAD~1: 첫 번째 부모 / HEAD~3: 3대 위
.gitignore: 새 파일의 추적 제외 규칙

03 브랜치와 태그

로컬 / 원격 / 전체 브랜치 목록

```
git branch
git branch -r
git branch -a
```

브랜치 생성 / 브랜치 전환

```
git branch <branch>
git switch <branch>
```

브랜치를 새로 만들고 바로 전환

```
git switch -c <branch>
```

현재 브랜치 이름 변경

```
git branch -m <new-name>
```

로컬 브랜치 삭제 / 강제 삭제

```
git branch -d <branch>
git branch -D <branch>
```

-d: 병합 여부 검사. -D: 검사 없이 강제 삭제.

현재 커밋에 버전 이름(태그) 부여

```
git tag <tag-name>
```

04 병합 (merge)

feature의 변경을 main에 병합

```
git switch main
git merge feature
```

병합 결과는 현재 브랜치(main)에 반영.

가능해도 fast-forward 대신 병합 커밋 생성

```
git merge --no-ff feature
```

변경을 합쳐 한 커밋으로 기록

```
git merge --squash feature
git commit -m "message"
```

--squash 자체는 커밋을 생성하지 않음.

충돌 해결 후 병합 마무리

```
git add <file>
git merge --continue
```

진행 중인 병합을 중단

```
git merge --abort
```

05 리베이스 (rebase)

커밋을 새 기준 위에 다시 적용. 재작성된 커밋은 ID가 바뀜.

feature 커밋을 main의 최신 커밋 위로 이동

```
git switch feature
git rebase main
```

최근 3개 커밋 순서·메시지·합치기 편집

```
git rebase -i HEAD~3
```

충돌 해결 후 계속 / 리베이스 중단

```
git add <file>
git rebase --continue
git rebase --abort
```

06 되돌리기

스테이징 취소 (파일 수정 내용은 유지)

```
git restore --staged <file>
```

스테이징하지 않은 변경을 index 버전으로 복원

```
git restore <file>
```

해당 파일의 스테이징하지 않은 변경을 버림.

과거 커밋을 확인 (브랜치에서 분리)

```
git switch --detach <commit>
```

읽기 전용이 아님. 작업을 유지하려면 새 브랜치 생성.

커밋의 변경을 취소하는 새 커밋 생성

```
git revert <commit>
```

기존 커밋 이력을 유지. 공유한 변경의 취소에 적합.

직전 커밋 취소 + 스테이징 상태 유지

```
git reset --soft HEAD~1
```

직전 커밋 취소 + 스테이징 해제

```
git reset --mixed HEAD~1
```

파일의 수정 내용은 유지. --mixed가 기본값.

직전 커밋 취소 + 파일도 이전 상태로 복원

```
git reset --hard HEAD~1
```

추적 파일의 변경을 잃음. 일부 미추적 파일도 없어질 수 있음.

merge / rebase / revert 구분

merge: 브랜치의 변경을 합침
rebase: 커밋을 새 기준 위에 재적용
revert: 변경을 상쇄하는 커밋 추가
공유한 커밋의 reset/rebase는 협업자와 조율.

07 상태와 이력 확인

수정·새 파일·스테이징 상태 확인

```
git status
```

커밋 이력을 간단히 / 그래프로 확인

```
git log --oneline
git log --oneline --graph --all
```

스테이징 전 / 스테이징된 변경 비교

```
git diff
git diff --cached
```

두 커밋 사이 변경 비교 / 커밋 상세 확인

```
git diff <commit1> <commit2>
git show <commit>
```

08 임시 보관 (stash)

추적 파일의 변경을 임시 보관

```
git stash push -m "message"
```

작업 디렉터리·스테이징 변경을 보관하고 정리.

미추적 파일 포함 / 무시된 파일까지 포함

```
git stash push -u
git stash push -a
```

변경의 일부만 골라 임시 보관

```
git stash push -p
```

보관 목록 / 선택한 보관 내용의 diff 확인

```
git stash list
git stash show -p "stash@{1}"
```

stash@{0}: 가장 최근. {1}: 그다음 항목.

적용하되 보관 항목 유지 / 성공 시 삭제

```
git stash apply
git stash pop
```

충돌이 나면 pop도 해당 항목을 유지.

특정 보관 항목 적용 / 삭제

```
git stash apply "stash@{1}"
git stash drop "stash@{1}"
```

모든 보관 항목 삭제

```
git stash clear
```

보관했던 변경을 잃을 수 있으므로 내용 확인.

09 원격 저장소 연결

origin 이름으로 원격 저장소 연결

```
git remote add origin <url>
```

원격 이름과 주소 확인

```
git remote -v
```

원격 이름 변경 / 연결 제거

```
git remote rename <old> <new>
git remote remove <remote>
```

remove는 로컬의 연결 설정을 제거.

10 동기화 (fetch / pull / push)

원격 이력 가져오기 (현재 브랜치 병합 없음)

```
git fetch origin
```

원격의 특정 브랜치 이력 가져오기

```
git fetch origin <branch>
```

원격 변경을 가져와 현재 브랜치에 병합

```
git pull --no-rebase origin main
```

main을 체크아웃한 상태에서 사용하는 예시.

원격 변경 위로 내 로컬 커밋을 재적용

```
git pull --rebase origin main
```

main을 체크아웃한 상태에서 사용하는 예시.

현재 브랜치 첫 push + 추적 관계 설정

```
git push -u origin <branch>
```

-u: 이후 pull-push의 기본 원격 브랜치를 설정.

추적 관계가 설정된 현재 브랜치 업로드

```
git push
```

특정 브랜치 / 태그 업로드

```
git push origin <branch>
git push origin <tag-name>
```

기존 checkout 명령어 대응

git checkout <branch> → git switch
git checkout -b <branch> → switch -c
파일 복원은 git restore로 구분.
Git은 로컬에서도 사용 가능.
GitHub은 원격 저장소를 제공하는 서비스.

오늘은 여기까지.

Thank you for listening.